

Inference Acceleration as Closed-Loop Perturbation: Closed-Loop Sensitivity-Guided Tactic Search for VLA Policies

Anonymous Authors

Abstract

Robot VLA policies are closed-loop controllers: an implementation change at one step can alter future observations, contact timing, task margins, and trajectory basins. We formulate post-training acceleration as closed-loop policy perturbation and show why static checks are incomplete: local action error is filtered by feedback dynamics, open-loop drift misses the accelerated policy’s induced state distribution, and sensitivity is anisotropic across action channels, rollout phases, and model boundaries. We propose Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS), which prunes acceleration tactics with cheap proxies, evaluates survivors with matched repair/regression rollouts and latency, and selects behavior-first, speed-constrained, or routed tactics on held-out folds. On GR00T N1.5 in LIBERO-10, speed-only action-head compilation gives a $2.23\times$ median serving-latency speedup on a 33-case probe but reduces success from 19/33 to 16/33; protecting only policy steps 0–120 restores 19/33 while preserving speed. Across two N1.5 30-case folds, 0–120 and a combined early-block plus 0–120 tactic both reach 47/60 pooled success, but favor different speed–risk objectives. A small GR00T N1.7 pilot check further suggests task-conditioned tactic routing as a high-speed trade-off point. The contribution is a robotics evaluation and design guide for acceleration under closed-loop sensitivity, not a claim of universal backend superiority.

1 Introduction

Large vision-language-action models increasingly serve as generalist robot policies. They map visual observations and language instructions into action chunks, often through diffusion-style denoising or transformer action heads [2–4, 7, 10]. As these models grow, deployment systems will quantize weights, compile graphs, fuse kernels, replay static CUDA graphs, cache repeated structure, or mix high- and low-precision execution. The usual question is whether the accelerated implementation is numerically close to the reference model and faster to serve.

For closed-loop robot policies, numerical closeness is not the whole problem. An action difference at time t changes the next observation; the next action is then produced on a state that may not have been visited by the original FP16 policy. In manipulation, this feedback can change contact timing, grasp stability, object pose, or

whether the terminal state crosses a binary success predicate. A tiny implementation difference can be absorbed in one phase, amplified near a contact boundary, or even repair a failure by nudging the rollout into a better trajectory basin.

This paper studies the following question: how should VLA inference acceleration be designed and evaluated when the accelerated implementation is part of a closed-loop control system? Our answer is that post-training acceleration should be treated as a structured policy perturbation. Quantization, graph compilation, eager islands, and kernel replacement are implementation maps from a reference policy to a constrained, faster policy. The relevant object is not only the one-step action drift on fixed observations, but also how the implementation redistributes success and failure over matched closed-loop rollouts.

We make three contributions:

1. We formulate VLA inference acceleration as a closed-loop policy perturbation problem and derive local propagation, margin-crossing, and distribution-shift expressions that explain why open-loop drift is not sufficient.
2. We provide a GR00T/LIBERO behavior study showing that equal-size action perturbations, rollout windows, and graph boundaries have unequal closed-loop consequences, so aggregate success and mean drift hide important behavioral churn.
3. We introduce Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS), a rollout-budget-aware procedure that estimates sensitive channels, phases, and graph boundaries, then selects behavior-first, speed-constrained, or routed acceleration tactics by held-out repair/regression and latency.

The contribution is a design-space map rather than a single deployment backend. We use controlled perturbation and compilation probes to expose which implementation changes are absorbed, amplified, repaired, or regressed, and which dimensions, phases, or modules deserve protection. The resulting rule is simple: identify closed-loop-sensitive regions, then choose acceleration tactics by held-out paired repair/regression and speed rather than by a single local probe.

2 Related Work

VLA policies. RT-1, RT-2, OpenVLA, and GR00T show that large vision-language-action policies can transfer visual-language knowledge into robot actions [2, 3, 7, 10]. Diffusion Policy and related action-diffusion methods generate actions through conditional denoising under receding-horizon control [4]. Our work studies what happens when such closed-loop policies are accelerated after training.

Post-training quantization and acceleration. LLM.int8(), GPTQ, SmoothQuant, and AWQ preserve transformer quality with calibration-aware quantization [5, 6, 8, 12]. QuantVLA adapts post-training quantization (PTQ) to VLA models with selective quantization and compensation [13]. These works motivate efficient backends; we study the complementary robotics question of how small acceleration-induced action differences change closed-loop rollout behavior.

Sequential prediction and feedback. DAgger shows how fixed-distribution evaluation can miss compounding closed-loop errors [11]. Feedback control likewise emphasizes controller perturbations, plant dynamics, and feedback gain [1]. We apply this lens to inference acceleration.

3 Closed-Loop Perturbation View

Let $\pi_\theta(a | s, l)$ denote a reference FP16 VLA policy conditioned on observation s and language l . A post-training acceleration transform constructs an implemented policy

$$\varphi = \mathcal{A}(\theta; C, S), \quad \pi_\varphi = \mathcal{A}(\pi_\theta; C, S), \quad (1)$$

where $\mathcal{A}$ may represent quantization, compilation, mixed precision, kernel replacement, or graph replay; C denotes calibration data; and S denotes systems choices such as graph boundaries or shape buckets. Even without gradient updates, this transform changes the implemented policy function.

3.1 Sensitivity Propagation

For local dynamics

$$s_{t+1} = f(s_t, a_t), \quad a_t = \pi(s_t, l), \quad (2)$$

let $\eta_t = \pi_\varphi(s_t, l) - \pi_\theta(s_t, l)$ be the local acceleration-induced action perturbation on the nominal FP16 trajectory. Let $A_t = \partial f / \partial s$, $B_t = \partial f / \partial a$, $K_t = \partial \pi_\theta / \partial s$, and $F_t = A_t + B_t K_t$. Under a local first-order approximation,

$$\delta s_T \approx \sum_{t=0}^{T-1} (F_{T-1} F_{T-2} \cdots F_{t+1}) B_t \eta_t. \quad (3)$$

Equation (3) uses the FP16 feedback Jacobian as a nominal proxy. If acceleration also changes policy gain, $K_t^\varphi = K_t + \Delta K_t$, then $\delta s_{t+1} \approx (F_t + B_t \Delta K_t) \delta s_t + B_t \eta_t$.

The surrogate omits hard-to-estimate ΔK_t ; closed-loop validation captures the combined rollout effect of offsets and gain changes empirically. Thus acceleration error is not determined by $\|\eta_t\|$ alone. Its effect depends on direction, dynamics, policy feedback, and future closed-loop gain.

3.2 Margin Crossing

Let $h(s_T)$ be an implicit terminal success margin, with success when $h(s_T) > 0$. For a small terminal perturbation,

$$h(s_T^\varphi) - h(s_T) \approx \nabla h(s_T)^\top \delta s_T. \quad (4)$$

A rollout can flip when

$$|\nabla h(s_T)^\top \delta s_T| \gtrsim |h(s_T)|. \quad (5)$$

The sign determines whether the perturbation repairs a failure or regresses a success. This explains why two rollouts with similar one-step action drift can have different outcomes: high-margin trajectories absorb perturbations, while low-margin trajectories near contact or placement boundaries can cross the success threshold.

3.3 Open-Loop Drift Is Insufficient

Offline validation estimates a loss $\ell(s)$ under a fixed distribution, typically d_{π_θ} or a dataset distribution. Closed-loop accelerated execution samples from d_{π_φ} :

$$\mathbb{E}_{d_{\pi_\varphi}}[\ell] = \mathbb{E}_{d_{\pi_\theta}}[\ell] + \left(\mathbb{E}_{d_{\pi_\varphi}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell] \right). \quad (6)$$

Open-loop action drift controls only the fixed-distribution term. It does not control how far the accelerated policy moves the visited state distribution. This motivates matched closed-loop rollouts and paired repair/regression accounting.

3.4 Anisotropic Sensitivity

Equations (3) and (5) motivate a sensitivity-weighted local surrogate

$$R(T_i) \approx \mathbb{E} \sum_t \eta_t(T_i)^\top G_t \eta_t(T_i), \quad (7)$$

where G_t summarizes downstream dynamics, feedback, and terminal-margin sensitivity. The exact G_t is difficult to estimate, but the expression identifies the search coordinates: action channels, rollout phases, and model boundaries need not be equally sensitive. A useful acceleration candidate should have high speed benefit and low sensitivity-weighted risk.

Algorithm 1 turns this diagnostic view into a rollout-budget-aware selection procedure. It is not an optimal-control solver; it is a way to spend closed-loop evaluation on tactics most likely to matter.

Algorithm 1: Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS)

1. **Input:** candidate tactics $\mathcal{T}$, matched probe rollouts $\mathcal{P}$, measured latency $L(T_i)$, and paired repairs/regressions $R^+(T_i), R^-(T_i)$ on each validation fold.
 2. Filter $\mathcal{T}$ by cheap systems and open-loop proxies: operator coverage, static shape compatibility, expected speedup, action drift, layer drift, and whether the tactic touches known sensitive phases or modules.
 3. Evaluate the surviving frontier on fragile matched rollouts. Record success, latency, first divergence, end-effector (EEF) drift, repairs, and regressions.
 4. Re-score the frontier on held-out folds. Let S_f , U_f , and R_f^- denote success, speedup, and paired regressions on fold f . Behavior-first selection uses the lexicographic rule ($\max \min_f S_f$, $\min \sum_f R_f^-$, $\max \min_f U_f$). Speed-constrained selection maximizes $\min_f U_f$ subject to $\min_f S_f \geq S_{\min}$ and $\sum_f R_f^- \leq \rho$.
 5. If no single tactic dominates, evaluate conditional selectors $g(z) \rightarrow T_i$ over task or trajectory regimes and validate the selector as another tactic.
- Output:* a behavior-first tactic, a speed-constrained Pareto tactic, or a validated tactic selector, with its matched success, regression, and latency report.

4 Experimental Setup

Model and benchmark. We use the GR00T N1.5 LIBERO long-horizon checkpoint and evaluate on LIBERO-10 [9, 10]. The accepted FP16 baseline matches the official reference result of 38/50 on the standard 5-trial LIBERO-10 run. The policy predicts action chunks with seven action components: x , y , z , roll, pitch, yaw, and gripper. Unless noted otherwise, experiments use 8 denoising steps.

Evidence tracks. Track A uses controlled action perturbations, first-divergence traces, and action-channel probes to identify sensitive output dimensions and rollout phases. Track B uses `torch.compile` action-head boundaries, eager islands, and duration fallbacks to test whether the sensitivity map can guide acceleration tactics. Track C evaluates held-out tactic selection on additional N1.5 folds and a small GR00T N1.7 check, including task-conditioned routing. Together, the tracks test whether closed-loop sensitivity can guide implementation choices before committing to a production backend.

Candidate tactics. The speed-oriented tactic compiles the action-head execution path. Protection tactics keep selected layer blocks in eager mode or route a fixed duration window through the eager implementation. These controlled tactics let us test the closed-loop selection rule without confounding it with a production custom kernel or low-precision backend.

Evaluation platform. Closed-loop rollouts run on a remote single-GPU NVIDIA RTX 5090 machine with LIBERO headless EGL. The simulator client and policy server run as separate processes on the same machine. Reported latency is server-side warm `get_action` p50, excluding model loading, prewarm, cold `torch.compile`/Inductor compilation, simulator stepping, and video I/O.

Closed-loop protocol. The sensitivity-guided acceleration search starts with a 33-case matched probe spanning tasks 4, 6, and 8, including easy successes, speed-only regressions, and beneficial speed-only branches. We then validate tactic selection on two additional N1.5 30-case folds over all LIBERO-10 tasks, using initial states 15–17 and 18–20. Finally, we run a small GR00T N1.7

held-out check on 15 task-initialization pairs over tasks 0, 1, 4, 6, and 8 with initial states 24–26. This last check tests whether the tactic-search workflow, not the specific 0–120 window, transfers to a newer checkpoint. Every mode is evaluated on the same task-initialization pairs with deterministic policy seeds and paired against the same FP16 baseline. We report success, paired repair/regression, server p50 latency, eager-request fraction, first-divergence diagnostics, worst-fold success, and worst-fold speedup.

5 Results

5.1 Not All Dimensions, Durations, or Layers Are Equal

To isolate sensitivity from any specific backend, we ran controlled probes on two FP16-success cases. All full-horizon single-channel probes use the same continuous-action perturbation norm, $\|\delta a\|_2 = 0.03$. Table 1 shows that equal-norm perturbations have very different consequences. A z perturbation is absorbed in both cases, while y and rotation perturbations are catastrophic. Time also matters: the same y perturbation is damaging early but can be absorbed later.

Table 1: Controlled action perturbations on two FP16-success cases. S/F denotes success/failure and policy requests.

Probe	Window	Outcomes
No perturbation	Full	S224; S649
6D continuous	Full	F991; F991
x only	Full	S349; F991
y only	Full	F991; F991
z only	Full	S239; S666
Roll only	Full	F991; F991
Pitch only	Full	F991; F991
Yaw only	Full	F991; F991
y task 4	0:75 / 75:150 / 150:225	F991; F991; S219
y task 6	0:200 / 200:450 / 450:700	F991; S697; S752

Layer and graph boundaries are also sensitive. Compiling the action-head model gives a $2.21\times$ speedup on two probe cases and succeeds on both, but leaving block 0 or block 1 as an eager island changes closed-loop outcomes

despite nearly identical p50 latency. The result cannot be explained by speed alone; it is a layer/boundary sensitivity effect.

Table 2: Layer/boundary sensitivity under action-head compilation.

Execution mode	Success	T4:i9	T6:i8	p50
FP16 baseline	2/2	S224	S649	160.53 ms
Compile action head	2/2	S222	S916	72.80 ms
Compile + block 0 eager	1/2	F991	S404	72.62 ms
Compile + block 1 eager	0/2	F991	F991	73.10 ms

Representative first-divergence traces show temporal separation: action differences appear first, millimeter-scale EEF differences appear later, and outcome flips occur much later through accumulated contact and progress changes. This supports the diagnostic principle: map sensitivity across action channels, rollout phases, and module boundaries before choosing what to compile, route, or protect.

5.2 Sensitivity-Guided Acceleration Search

We next test whether the sensitivity map can guide an engineering choice. The speed-oriented candidate compiles the action head. Protection candidates either keep selected layer blocks in eager mode or use a duration window in which the policy falls back to eager execution. Table 3 summarizes the 33-case discovery probe.

Table 3: Layer and duration proxy search on the 33-case matched set. Server p50 is warm serving latency.

Mode	Success	p50	Speedup	Eager frac.
FP16 baseline	19/33	156.22 ms	1.00×	—
Speed-only compile	16/33	70.20 ms	2.23×	—
Blocks 0–3 eager	18/33	67.58 ms	2.31×	—
Duration 0–250	18/33	88.75 ms	1.76×	0.376
Duration 0–120	19/33	69.66 ms	2.24×	0.190
Duration 0–180	18/33	77.55 ms	2.01×	0.281
Duration 0–220	16/33	90.49 ms	1.73×	0.328
Duration 80–240	14/33	75.98 ms	2.06×	0.214
Duration 120–280	16/33	73.48 ms	2.13×	0.216
Duration 160–240	15/33	69.29 ms	2.25×	0.111
Duration 240–320	17/33	66.71 ms	2.34×	0.084

The key probe result is not that more eager execution is safer. Speed-only compilation is fast but loses three successes relative to FP16. A broad early duration fallback, steps 0–250, recovers two successes but costs latency. The finer sweep reveals a better local point: protecting only steps 0–120 restores FP16-level success, 19/33, while preserving speed-only latency. Longer windows are worse, so the sensitive region is narrow and early. Small p50 inversions among compiled variants reflect warm-serving measurement noise and different eager/compiled call mixes; we use latency as a frontier coordinate rather than treating any single p50 value as exact.

Paired repair/regression explains the aggregate. Relative to speed-only, the 0–120 window repairs seven cases

and regresses four, for a net +3. Relative to FP16, it repairs two baseline failures and regresses two baseline successes. Thus the accelerated implementation is not simply restored to the FP16 point; it explores a nearby closed-loop solution space.

Table 4: Paired decomposition versus speed-only compilation.

Mode	Repair	Regress	Net
Duration 0–250	4	2	+2
Duration 0–120	7	4	+3
Duration 0–180	5	3	+2
Duration 0–220	4	4	0
Duration 80–240	3	5	-2
Duration 120–280	2	2	0
Duration 160–240	2	3	-1
Duration 240–320	2	1	+1

Table 5: Multi-fold tactic selection. Fold A uses init 15–17; Fold B uses init 18–20. Regressions are against FP16.

Tactic	A	B	Pool	WSucc	MSpd	WSpd	Reg
Speed	25/30	20/30	45/60	0.667	2.01×	1.75×	7
0–120	22/30	25/30	47/60	0.733	1.84×	1.71×	3
Combo	22/30	25/30	47/60	0.733	1.41×	1.07×	1

Held-out folds change the engineering conclusion. Speed-only is best on Fold A but regression-prone on Fold B. The combined tactic has the fewest regressions, but its worst-fold speedup is only 1.07×. The 0–120 tactic is therefore preferable under a speed-constrained objective, while the combined tactic is preferable under a behavior-first objective. Sensitivity-guided acceleration should thus output a Pareto choice under a validation protocol, rather than a single protected region selected from one probe.

Table 6: N1.7 held-out transfer check on 15 task-initialization pairs. Repairs and regressions are paired against FP16.

Tactic	Success	p50	Speedup	Repair / regress
FP16	13/15	110.40 ms	1.00×	—
Speed-only	11/15	74.79 ms	1.48×	1 / 3
Window 0–20	14/15	100.98 ms	1.09×	2 / 1
Routed tactic	13/15	79.07 ms	1.40×	2 / 2

The N1.7 transfer check keeps the focus on the procedure rather than the 0–120 window. The fastest tactic has the most regressions, the conservative window has the best held-out success but little speedup, and task-conditioned routing preserves most of the speed-only gain while still requiring held-out validation. Figure 1 visualizes this speed-risk frontier.

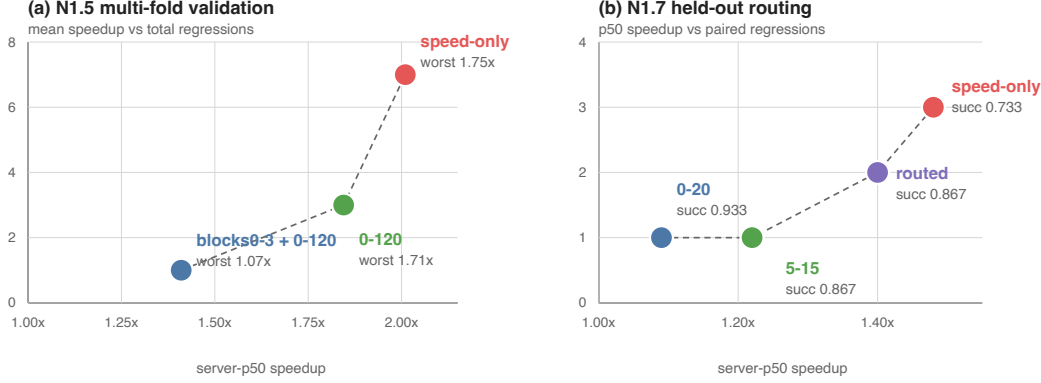


Figure 1: Speed-risk Pareto view of held-out tactic selection. Left: N1.5 multi-fold validation. Right: N1.7 held-out validation, including a task-conditioned tactic router.

6 Discussion

Why can acceleration improve a rollout? Repairs are not paradoxical under the perturbation view. FP16 is a learned policy with fragile regions, so a small implementation perturbation can move a rollout away from one failure basin and into another basin that satisfies the benchmark predicate. Aggregate gains therefore need paired accounting: a tactic can improve the total count while still regressing individual FP16 successes.

Using CLSG-TS. Algorithm 1 summarizes the workflow. Start from a speed-oriented candidate, measure latency and matched closed-loop success, decompose outcomes into paired repairs and regressions, identify sensitive action channels, rollout windows, and graph boundaries, then re-evaluate a small tactic frontier on held-out folds. The output is not a universal protected region, but a deployment choice on the Pareto frontier: behavior-first, speed-constrained, or conditionally routed.

Sensitivity-guided implementation search. This is analogous to compiler tactic search, but with a behavioral validation target. A compiler can enumerate many operator implementations for a fixed shape, benchmark latency, check local numerical agreement, and choose the fastest valid tactic. VLA acceleration has a similar candidate space—quantization scopes, graph boundaries, eager islands, fallback windows, kernel replacements, and calibration choices—but each candidate must be judged by closed-loop risk. Exhaustive rollout evaluation is infeasible. The useful strategy is therefore to prune candidates with cheap static and open-loop proxies, run small matched proxy rollouts on fragile cases, and reserve larger held-out validation for the few candidates with the best speed/risk trade-off. In this reading, 0-120 is a sensitivity-pruned candidate, not a universal tactic.

Task-conditioned tactic routing. The N1.7 check suggests a deployment need not use one global acceleration tactic for every task. A lightweight router can select

among validated inference tactics by task identity or trajectory regime. This is reminiscent of mixture-of-experts routing, but the routed objects are implementation tactics, not learned model experts. The same validation rule applies: routing is useful only when its repair/regression profile is measured on held-out paired rollouts.

Limitations. The current evidence is behavior-level rather than backend-complete. The mixed-execution probes use prototype `torch.compile` boundaries and eager fallbacks; production TensorRT, CUDA-graph, or custom C++/CUDA runtimes may impose extra graph-break, routing, memory, and scheduling costs. Results are centered on one GR00T N1.5 checkpoint and LIBERO-10, with a small N1.7 held-out check rather than a broad model-family benchmark. The 33-case discovery probe, two 30-case N1.5 held-out folds, and 15-case N1.7 check expose tactic instability but are not statistically exhaustive. Online sensitivity-triggered routing, production-backend profiling, larger task suites, and real-robot validation remain future work.

7 Conclusion

Post-training inference acceleration of VLA policies should be viewed as closed-loop policy perturbation. Small action errors matter when they interact with feedback dynamics, task margins, and trajectory basins. In our GR00T/LIBERO study, speed-only action-head compilation gives substantial latency gains but introduces regressions; sensitivity-guided duration fallbacks and routed tactics recover better speed-risk trade-offs. CLSG-TS turns these observations into a deployment workflow: discover sensitive regions, evaluate candidate tactics with matched repair/regression, and select a behavior-first, speed-constrained, or routed point on the held-out speed-risk frontier.

Beyond tactic selection, CLSG-TS can seed robust training: noise-aware distillation or reinforcement learn-

ing fine-tuning could perturb the states, phases, channels, and graph boundaries that matter most in closed loop. VLA acceleration papers should report paired rollouts, sensitivity probes, held-out tactic validation, and latency rather than only open-loop drift or aggregate success.

References

- [1] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL <https://fbsbook.org>.
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. URL <https://arxiv.org/abs/2212.06817>.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL <https://arxiv.org/abs/2303.04137>.
- [5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022. URL <https://arxiv.org/abs/2208.07339>.
- [6] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL <https://arxiv.org/abs/2210.17323>.
- [7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. URL <https://arxiv.org/abs/2406.09246>.
- [8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL <https://arxiv.org/abs/2306.00978>.
- [9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL <https://arxiv.org/abs/2306.03310>.
- [10] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [11] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *arXiv preprint arXiv:1011.0686*, 2010. URL <https://arxiv.org/abs/1011.0686>.
- [12] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. URL <https://arxiv.org/abs/2211.10438>.
- [13] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026. doi: 10.48550/arXiv.2602.20309. URL <https://arxiv.org/abs/2602.20309>.